Software Engineering Project

DispatchNet:

Alessio Zanon
Leonardo Tonetta
Luca Fiorini

User Flow & Implementation Analysis

Contents

Readability notes

Bold words are system-important constructs.
Underlined nouns are actors of the system.
Italic words are special keywords to denote important limits of the project.
Hyperlinks in reference to sections and subsections will be blue.
Hyperlinks to external web content will be in magenta.

Purpose

This document describes all the relevant information regarding the development of Project "TRE", which implements a rail transport management system.

It focuses on our current mock implementation and provides considerations towards it's current architecture and how it relates to a ideal final product.

Specifically, we outline:

- User flows related to the actions performed by all users.
- The Class Diagram of our current implementation.
- The ideal properties of the final implementation's API.
- Internal Interfaces that aim to mimic the behavior real-world external APIs.
- The documentation of the implemented processes.
- Important implementation decisions taken.

Chapter 1

Previous deliverables

Original, up-to-date files of D1 and D2 available at our Github here and Github here respectively

Chapter 2

D3

RE: Objectives

A major European rail infrastructure manager received an idea for a unified rail service scheduling and ticketing system, and has chosen us to prototype a trial run, over multiple regions in their network.

Thus, the project provides a platform that covers the needs of both passengers and train companies, as well as an underlying management system of any regional-sized rail network, which supports network managers in its administration.
It provides a set of functionalities which support the planning of train travel. The system lets passengers discover paths, view scheduling and manage tickets. On the other side, it lets train companies introduce train services and network managers administer the network.
The project coordinates the functionalities for each stakeholder into a solution that has been identified in an application accessible for the aforesaid users.

The main objectives of the project are:

Ticketing system

Subsystem which deals with tickets, from the search to the possible actions after the purchase. It includes basic operations that the passenger can perform, such as:

- Viewing tickets details.
- Purchase.
- Accessing tickets history.
- Cancellation.

Route-finding

The system shall let passengers select a starting station, a destination station, and either a departure time or arrival time.
It computes and ranks possible **routes** that best fit the user's request and permissions (for example, not permitting cargo trains to passengers).
The user must be able to compare the options based on:

- Departure and arrival times.
- Number of transfers.
- Possible transfer locations.

Virtual board

The system provides passengers access to departure and arrival boards for any station. It also shows intermediate stations and scheduled times for each individual train service.

Rail system modeling

The system provides a data-driven approach to rail network modelling, supporting an efficient administration of the infrastructure.
It shall support:

- Management of the network by the network manager.
- Management of rolling stock by the network manager.
- Submission of a new service request by train companies.
- Review of a service request by the network manager.

RE: Stakeholders

Internal

Human

Network managers:, the product owner. They manage the rail network combining the availability of train companies with passengers' needs.
They are the primary stakeholders for making functional decisions.

Non-Human

Database, where all the information used by the application are stored.

External

Human

Passengers, the main end users. They want to streamline their interactions with the train transport system, including:

- Searching for routes and schedules.
- Analyzing departure/arrival boards as well as train information.
- Purchasing tickets.

Train companies, who provide the rolling stock. They request the introduction of services. They are also expected to provide information of trains they ask to be managed.
Anonymous users, who have not yet authenticated. They are considered as passengers with restricted access to functionalities that require authentication.

Non-human

Payment gateway, which provides the underlying payment methods.

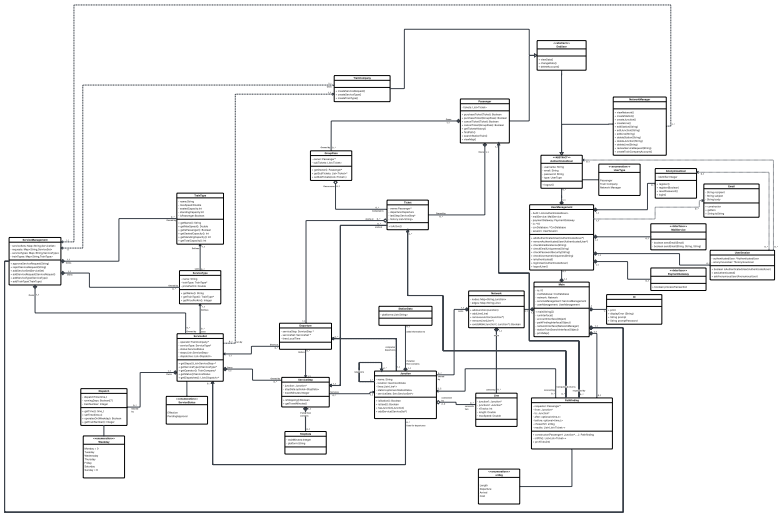
2.1 Use case implemented

UC1-12.1, 13-32

So we didn't implement: UC12.2

2.2 Class Diagram

In the following image, the updated class diagram, changed in order to take advatantage of the Java language and to accomodate the implementation.



Original image can be found on our Github here

We introduced the following classes:

- Email: used by the UserManagement class to send an Email through a MailService;
- MailService: interface defining the necessary methods for a real MailService, such as the MockMailService implemented in the code;
- PaymentGateway: interface defining the necessary methods for a real PaymentGateway, such as the MockPayementGateway implemented in the code;

- UserSession: for simplicity we changed to focus of the application on one user at a time. This class is used to keep track of the current user that is using the program;
- IO: class used to group function to interact with the user;
- CsvDatabase: not shown because ideally it is implemented with an external SQL server.
- Location: merged two separate variables, latitude and longitude

We added attributes concerning the classes that have been introduced in UserManagement, removed getter and setter for better readability.
ToString() methods were intended as function to display the information, which has been implemented within the code, so not implemented explicitly. Some List types have been substituded by Map types because better representative.

The updated descriptions of the classes can be found on the Documentation.

2.3 Project Structure

In the following is shown the structure of the project.

Original image can be found on our Github here

It is composed of:

- the main modules:
  - departure: a specific service departing from a junction at a given time.;
  - infrastructure: the network and its management;
  - main: contains the runnable class, which implement the interface;
  - path_finding: the algorithm to find a path between stations;
  - service_management: the management of services;

  - ticketing: managemet of tickets information;
  - user_management: the management of users and their authentication.

- secondary tools:
  - csv_database: the actual database and its management;
  - IO_operations: input/output operation to interact with the user from terminal;
  - mail_service: integration with a possible mail service, as well as a temporary mock one;
  - payment_gateway: integration with a possible payment service, as well as a temporary mock one.

2.4 Architecture

The project is intended, in its platonic ideal, as a Client-server structure, where the client shoulders only the user interface and requests to the server, who instead is responsible for the actualy interfacing with the database and computing the responses.

Thanks to our chosen Implementation Language, the Client may be nearly any device capable of general computing, and thus we want to minimize the client-side device requirements to fully exploit our great portability.

Thus, the client is expected to perform only to the relatively light work of displaying responses and forming requests with user input to send the central server.

Not beholden to such limitations, the central server must then receive requests and perform the computatonally expensive operations, chief among them the weighted-graph-traversal over the train nework, but also loading and handling the information from the database.

Due to both skill and time limitations, the project is implemented as a temporary measure in a monolithic form, while still taking care to expect and enforce the necessary separation and operations for a hypothetical future division and upgrade to a client-server system ready for deployment.

2.5 Implementation Language

We chose **Java** for the operational part.

Our motivation for **Java** was the extreme portability and the very module-friendly structure, allowing us to experiment and tinker with different implementations of the same expected behaviour in a plug-and-play fashion. As described in the Team roles we splitted the work in modules from the beginning, so the modularity of **Java** came natural. The garbage collector feature of **Java** also ensures us to spend more time working of the functioning of our systems rather than the management of their lifetimes.

2.6 Documentation

Doxygen was chosen due to its in-built support for **Java** development and for its low overhead cost during code development, as the comment-tag system is very time efficient, while still being very sight-readable in the code files.
It is available at our github at the following link.

2.7 Project Dependencies

- JDK v25.0.3 or above

2.8 Database

To save data between different session we used a database saved as several CSV files.
It will be loaded at startup, containing at least the account of the network manager, with default information. The data will be managed within the code and saved to the database only at the end of the program.
Therefore, the number of entries for each file is variable. Wherever a list was needed, we used the convention of separating with ";" the elements of the list, which is put within a single field.

The database contains one file for each set of data:

- authenticated users,

- dispatches,
- junctions,
- lines,
- service_sets,
- service_types,
- station data,
- tickets,
- train_types.

The following files can be found for better readability in GitHub
The authenticated user contains data used for authentication and functionaility selection based on the type of user. It contains the following fields:

```
id,username,email,password,userType
0e3c90bc-0861-4019-8e82-8b56c0275106,Admin,admin@dispatchnet.it,VerySecurePassword1234,NetworkManager
```

Original image can be found on our Github here

The dispatch contains information to identify services and schedule it within a week .
The field runningDays is a list. It moreover contains the following fields:

```
serviceSetId,trainNumber,dispatchTime,runningDays
75a4546d-6b6b-41c3-b823-ae25be53637e,1001,05:04,WEDNESDAY;TUESDAY;FRIDAY;THURSDAY;MONDAY;SATURDAY
```

Original image can be found on our Github here

The junction contains data of the physical junction or station (considered as junction).
The field StationDataId is optional. It contains the following fields:

```
id,name,latitude,longitude,stationDataId
bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,Trento,46.072194035019535,11.118837720509594,699e0587-7686-473c-9e72-70f1de3361a1
```

Original image can be found on our Github here

The line contains data used to know the length and service consistency checks. It contains the following fields:

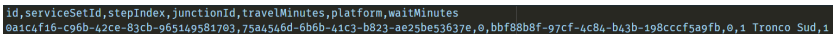
```
id,junction1Id,junction2Id,lengthMeters,maxSpeedKpH,nTracks
84a99b0c-fc4b-4338-9af1-5568cb80b8cf,bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,1ef67fd5-9fb3-4315-9125-5394ef91a75e,3100,50,1
```

Original image can be found on our Github here

The service set contains information related to its status. It contains the following fields:

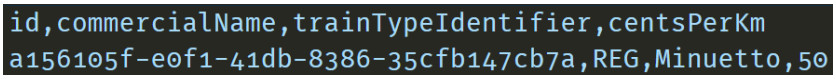
Original image can be found on our Github here

The service step contains data related to one step of the service.
The field platform is a list. It contains the following fields:



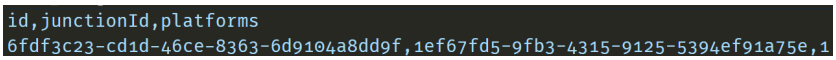
Original image can be found on our Github here

The service type contains the following fields:



Original image can be found on our Github here

The station data contains details of a station.
The field platform is a list. It contains the following fields:

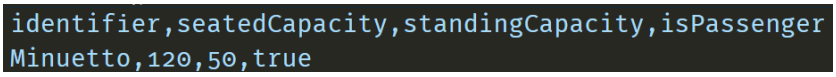


Original image can be found on our Github here

The ticket contains the following fields:

Original image can be found on our Github here

The train type contains information related to the physical train. It contains the following fields:



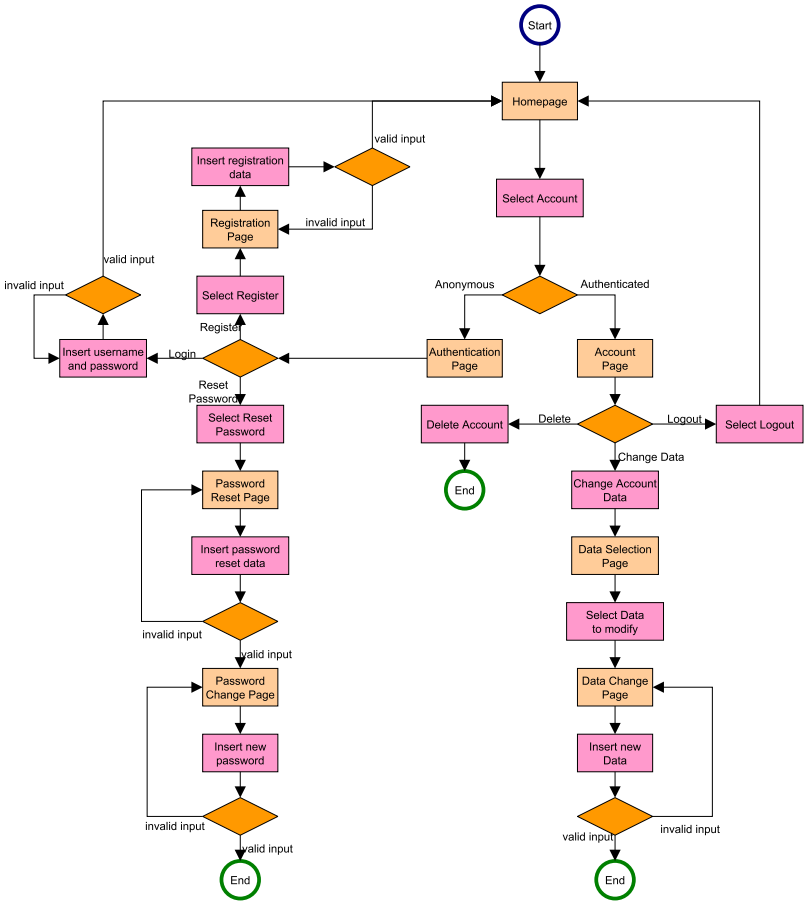
Original image can be found on our Github here

2.9 User Flows

2.9.1 User Management

The following user diagram describes the actions related to the user management and user authentication, common to all the users.

It provides different pages depending on the type of user. An authenticated user (I.E. passenger or train company, not network manager) can delete, view data (Account Page), change data or logout (also the network manager).
An anonymous user can register, reset its password or login.



Original image can be found on our Github here

UC implemented - corresponding function:

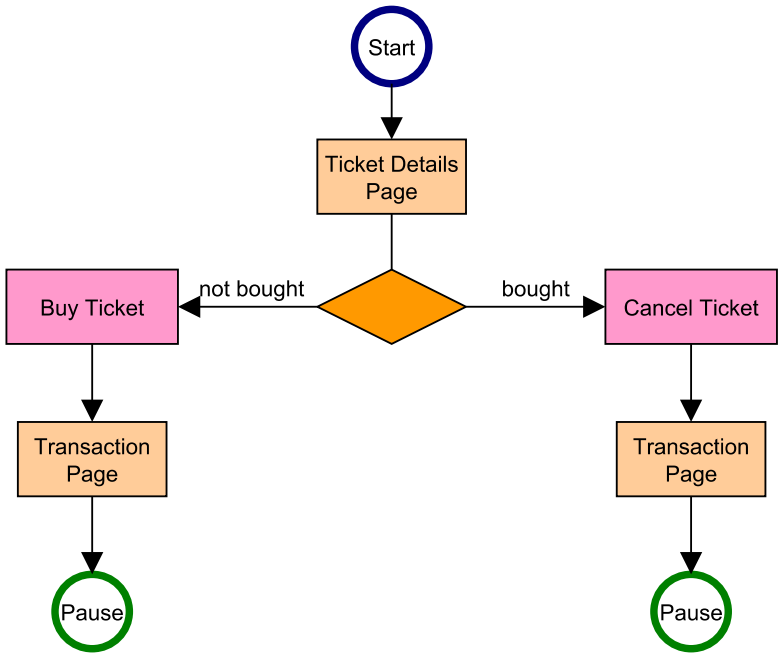
- UC1 - register

- UC2 - deleteAccount

- UC3 - viewData this is not explicit in the diagram because is in the Account Page
- UC4 - changeData
- UC5 - resetPassword
- UC6 - login
- UC7 - logout

2.9.2 Ticketing

The following user diagram describes the flow of the ticketing component, started from the Passenger's diagram.



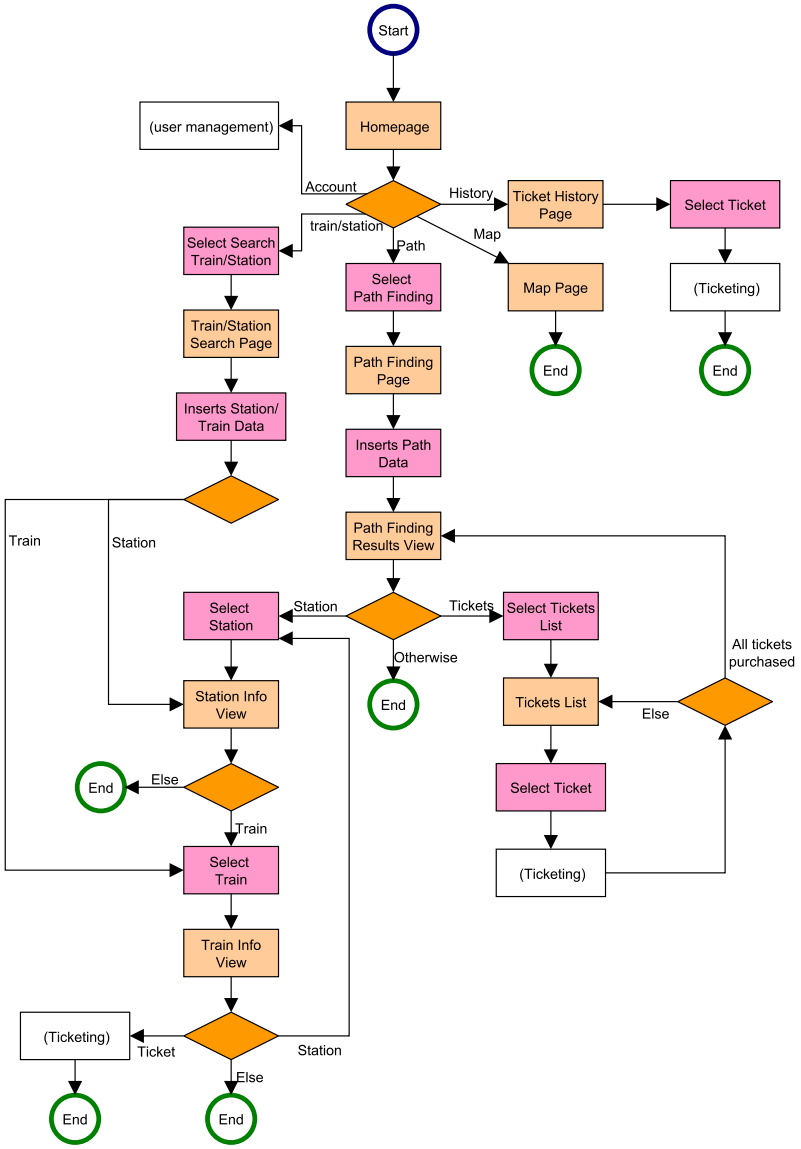
Original image can be found on our Github here

UC implemented - corresponding function:

- UC8 - implemented as buildDescription, used in other functions
- UC9 - purchaseTicket
- UC10 - cancelTicket
- UC11 - getTicketsHistory

2.9.3 Passenger

The following user diagram describes the actions the passenger can take. It can go to User Management, view its ticket history, search for a train or station or start a path finding procedure. In the last two it can move to the Ticketing flow.



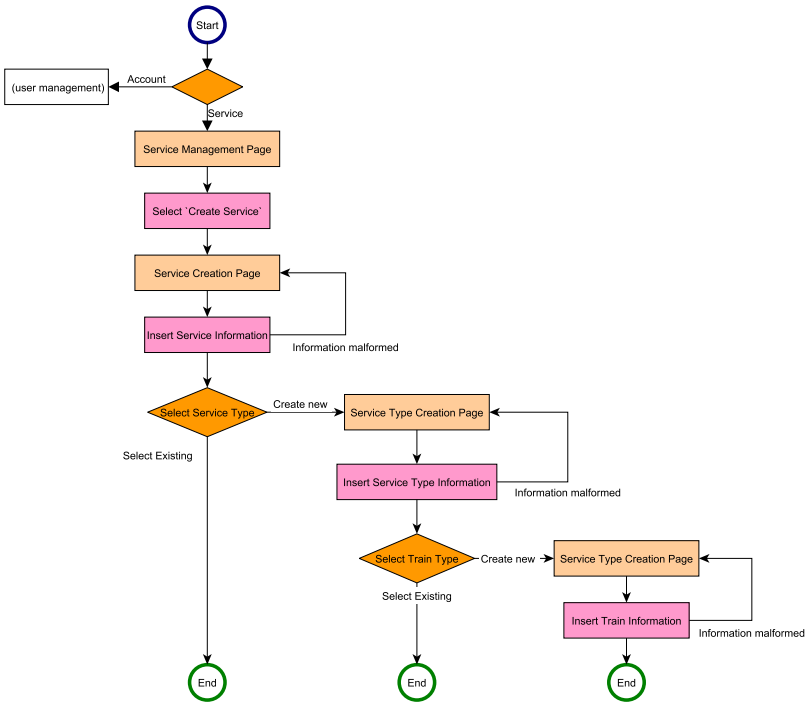
Original image can be found on our Github here

UC implemented - corresponding function:

- UC12 - PathFinding constructor
- UC12.1 - prchTcks
- UC13 - stationTrainSearchInterface
- UC14 - stationTrainSearchInterface
- UC15 - printMap

2.9.4 Train Company

The following user diagram describes the actions the train company can take. It can go to User Management or to the service management.



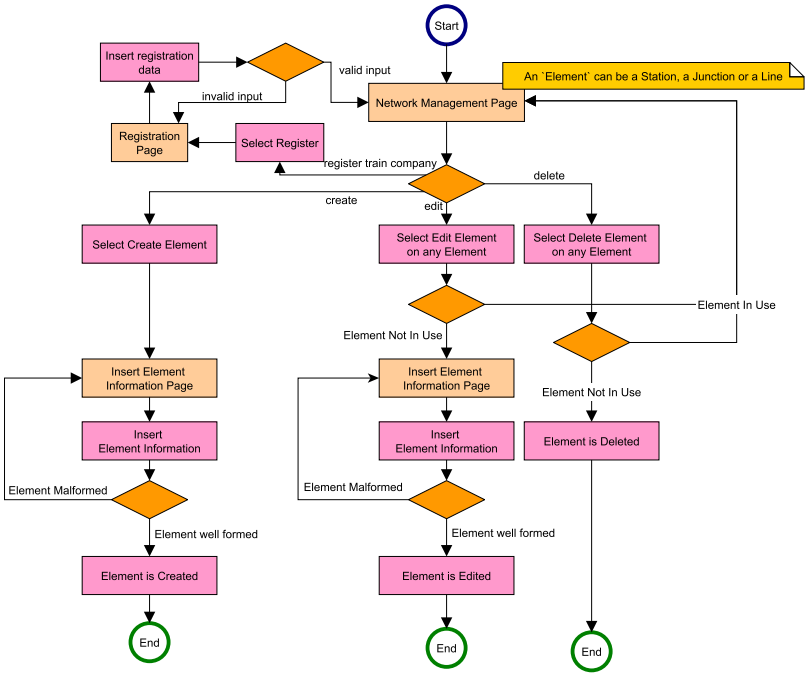
Original image can be found on our Github here

UC implemented - corresponding function:

- UC29 - createTrainType
- UC30 - createServiceType
- UC31 - createServiceRequest

2.9.5 Network Manager

The following user diagram describes the actions the network manager can take: managing the network and its elements ore creating a new account for a train company.



Original image can be found on our Github here

UC implemented - corresponding function:

- UC16 - viewNetwork
- UC17 - insertStationData
- UC18 - insertJunctionData
- UC19 - insertLineData
- UC20 - createStation
- UC21 - createJunction
- UC22 - createLine

- UC23 - editStation
- UC24 editJunction
- UC25 - editLine
- UC26 - removeStation
- UC27 - removeJunction
- UC28 - removeLine
- UC29 - serviceRequestReview

2.10 APIs

As this is a mock implementation, no real API has been implemented. Instead, we opted to build fake endpoints in the form of methods, so we could test the validity of our underlying data model. All client-facing operations are found within the `UserManagement` package, in the form of class methods grouped by their relevant user. The final implementation is planned to be a REST API and will adhere to the OpenAPI standard.

External APIs have been modeled as interfaces with mock implementations. The interface paradigm allows us to use valid invocations towards a fake implementation that can easily be swapped at a later date by a real implementation. Specifically, we have implemented the following fake external APIs

- `mail_service`: The mail service
  - Interface: `MailService`
  - Mock implementation: `MockMailService`
- `payment_gateway`: The payment gateway
  - Interface: `PaymentGateway`
  - Mock implementation: `MockPaymentGateway`

The mock implementation of payment gateway also has non-deterministic behavior allowing us to test adequate handling of real-world failure conditions.

2.11 GitHub repo

All the codes, images and documentation can be found at our GitHub

2.12 Code Preview

```java
public void login() {
    String username, password;
    boolean authenticated;

    // ask username and password until they match an existing user account
    do {
        username = userManagement.prompt("Enter username:");
        password = userManagement.promptPassword("Enter password:");

        authenticated = userManagement.authenticateUser(username, password);
        if (!authenticated) {
            userManagement.displayError(
                "Invalid username or password. Please try again."
            );
        }
    } while (!authenticated);

    // log the user in the account
    AuthenticatedUser user = userManagement.getAuthenticatedUserByUsername(username);
    userManagement.loginUser(user);
}
```

Original image can be found on our Github here

The *login* function allows Anonymous users to elevate their role. This is done by inserting username and password (using a dedicated IO operation that doesn't display inserted text on the display).

```java
public void register(boolean fromNetworkManager) {
    String username, email, password1, password2;

    // ask username until username is unique
    boolean usernameUnique;
    do {
        username = userManagement.prompt("Enter username:");
        usernameUnique = userManagement.checkUsernameUniqueness(username);
        if (!usernameUnique) {
            userManagement.displayError(
                "That username is already taken. " +
                "Please choose a different username."
            );
        }
    } while (!usernameUnique);

    // ask email until email is unique
    boolean emailUnique;
    do {
        email = userManagement.prompt("Enter email:");
        emailUnique = userManagement.checkEmailUniqueness(email);
        if (!emailUnique) {
            userManagement.displayError(
                "That email is already registered. " +
                "Please use a different email address."
            );
        }
    } while (!emailUnique);

    // if registering from NetworkManager, skip password input and use a default password
    if(fromNetworkManager) {
        password1 = "DefaultPassword1";
    }
    else {
        // ask password until password is confirmed and meets security requirements
        while (true) {
            password1 = userManagement.promptPassword("Enter password:");
            password2 = userManagement.promptPassword("Confirm password:");

            if (!password1.equals(password2)) {
                userManagement.displayError(
                    "Passwords do not match. Please try again."
                );
                continue;
            }

            if (!userManagement.checkPasswordSecurity(password1)) {
                userManagement.displayError(
                    "Password does not meet security requirements." +
                    " Please choose a stronger password."
                );
                continue;
            }

            break;
        }
    }
```

Original image can be found on our Github here

The *register* function is responsible for converting Anonymous users into *Passengers* upon submission and verification of login information, and email address. It checks that both email and username be unique, and the passowrd is both strong and inserted correctly (by matching to a second insertion).

```java
protected boolean authenticateUser(String username, String password) {
    for (AuthenticatedUser user : authenticatedUsers) {
        if (user.getUsername().equals(username) &&
            user.getPassword().equals(password)) {
            return true;
        }
    }

    return false;
}
```

Original image can be found on our Github here

The *authenticateUser* function matches the given login info with the one stored in the system, ensuring "wrong" login attempts fail and "right" ones complete successfully.

```java
public boolean isUsed() {

  List<Junction> junctions = List.of(this.junction1,this.junction2);

  return junctions.stream().anyMatch(thisJct -> { //check for both directions at both junctions
    Junction otherJct = junction1 != thisJct ? junction1 : junction2; //flip the direction

    return thisJct.getServices().stream().anyMatch(service -> {
      boolean hitThis = false;
      boolean hitOther = false;

      for (var step : service.getSteps()) {
        if (step.getJunction() == thisJct) {
          if (hitOther) {
            return true;
          }
          hitThis = true;
          hitOther = false;
        } else if (step.getJunction() == otherJct) {
          if (hitThis) {
            return true;
          }
          hitOther = true;
          hitThis = false;
        } else {
          hitOther = false;
          hitThis = false;
        }
      }
      return false;
    });
  });

}
```

Original image can be found on our Github here

Safety-checking function, it assures that a junction to be operated-upon does not have any services running on it, that may be disrupted or engage in undefined behaviour. This method checks whether a line is currently in use, this is not trivial unlike for Junctions where we simply need to check if the number of services that call there is above 0. To perform this check, I scan the services through both junctions in both directions (thisJct and otherJct

swap) if any service is found to traverse both junctions one directly after the other (in any order) anyMatch fails and the whole functions returns true.
It implements UC25, UC28

```java
class PathComparator implements Comparator<ArrayList<Ticket>> {
  private ArrayList<Comparator<ArrayList<Ticket>>> Comps;
  private srtAlg chosenSrt;

  /**
   * @brief Comparator constructor
   *
   * @details Initializes the sub-comparators, and sets the preferred one via the enum
   * @param alg The preferred sorting enum
   */
  public PathComparator (srtAlg alg) {
    this.Comps = new ArrayList<Comparator<ArrayList<Ticket>>>();
    this.Comps.add(new sortByLenght());
    this.Comps.add(new sortByDeparture());
    this.Comps.add(new sortByArrival());
    this.Comps.add(new sortByCost());
    this.chosenSrt = alg;
  }

  @Override
  public int compare (ArrayList<Ticket> L1, ArrayList<Ticket> L2) {

    int sortVal = 0; //Stores the return values
    switch (chosenSrt) { //Start with the chosen sorting mehtod
      case Length:
        sortVal = Comps.get(0).compare(L1, L2);
      break;

      case Departure:
        sortVal = Comps.get(1).compare(L1, L2);
      break;

      case Arrival:
        sortVal = Comps.get(2).compare(L1, L2);
      break;

      case Cost:
        sortVal = Comps.get(3).compare(L1, L2);
      break;

      default:
        //How did we get here?
    }

    for (int i = 0; i<4 && sortVal == 0; i++) { //If the sorth is inconclusive, try all the others
      sortVal = Comps.get(i).compare(L1, L2);
    }

    return sortVal;
  }


}
```

Original image can be found on our Github here

The overarching comparator that allows path sorting. It involves several sub-comparators, so that different preferences can be reflected (wanting a cheaper rather than strictly shorter path). First, it attempts to use the preferred sorting system, but failing that it iterates on them all to reduce the likelihood of draws. This way, even if two paths are equally short, the otherwise-preferrable one will have priority, without upsetting the overall

rankings.
It implements UC12

```
class sortByLenght implements Comparator<ArrayList<Ticket>> {
  public int compare (ArrayList<Ticket> L1, ArrayList<Ticket> L2) {

    LocalTime startL1 = L1.get(0).getDeparture().time();
    LocalTime endL1 = L1.get(L1.size()-1).getDeparture().time();
    int offsetL1 = L1.get(L1.size()-1).getLastStep().getTravelMinutes();

    LocalTime startL2 = L2.get(0).getDeparture().time();
    LocalTime endL2 = L2.get(L2.size()-1).getDeparture().time();
    int offsetL2 = L2.get(L2.size()-1).getLastStep().getTravelMinutes();

    long diffL1 = MINUTES.between(startL1, endL1.plusMinutes(offsetL1));
    long diffL2 = MINUTES.between(startL2, endL2.plusMinutes(offsetL2));

    //Shorter one is better
    if (diffL1 < diffL2) return -1;
    if (diffL2 > diffL2) return 1;

    //Compare Km lenght
    int metersL1 = 0;
    for (int i = 0; i<L1.size(); i++) {
      metersL1 += L1.get(i).getLength();
    }

    int metersL2 = 0;
    for (int i = 0; i<L2.size(); i++) {
      metersL2 += L2.get(i).getLength();
    }

    if (metersL1 != metersL2) return metersL1-metersL2;

    //If times and lenght are the same, continue with size comparison
    int size1 = L1.size();
    int size2 = L2.size();

    return size1-size2;
  }
};
```

Original image can be found on our Github here

An example of the sub-comparators invoked in the main path comparator. It abides by Java's standard for comparators, returning a negative integer if the first element is to be put earlier, zero if they are equal and a postitive integer if the latter is. It comparator specifically operates by, in order, attempting to find the quickest path (by comparing departure and arrival times), then the shortest (space-wise) and lastly the one involving the least exchanges.
It implements UC12

```java
public void prchTcks() throws Exception {

  Passenger ticketUser = (this.requester instanceof Passenger)? (Passenger) this.requester: null;
  if (!(this.requester instanceof Passenger)) throw new Exception("BadRequester");  //Guarantee user is Passenger

  StringBuilder queryText = new StringBuilder();
  queryText.append("Please select one of the following by number index\n-------\nq) Quit\n-------\n");

  //Load Tickets for selection
  for (int i = 0; i<results.size(); i++) {
    queryText.append(i);
    queryText.append(":\t");
    for (int ii = 0; ii<results.get(i).size(); ii++) {
      queryText.append(results.get(i).get(ii).getJunction().getName());
      if (ii+1<results.get(i).size()) queryText.append(", ");
    }
    queryText.append(".\n");
  }

  int index = 0;
  UserManagement cmd = Main.getUserManagement();
  String userReply = cmd.prompt(queryText.toString());

  if (userReply == "q") return; //Allow the user to quit

  index = Integer.parseInt(userReply);
  if (index < 0 || index >= results.size()) { //Index bound check;
    cmd.displayError("Invalid index, returning to menu");
    return;
  }

  boolean allPurchased; //Repetition flag

  do {
    allPurchased = true;
    queryText = new StringBuilder();
    queryText.append("Please select one of the following by number of index\n-------\nq) To quit\n-------\nq"); //Load tikets in chosen path to display
    for (int i = 0; i<this.results.get(index).size(); i++){
      queryText.append(i);
      queryText.append(":\t");
      queryText.append(this.results.get(index).get(i).getDescription());
      if (this.results.get(index).get(i).isActive()) queryText.append("\t Purchased!");
      else allPurchased = false;
      queryText.append("\n");
    }

    userReply = cmd.prompt(queryText.toString());

    if (userReply == "q") return;
    int ticketSelector = Integer.parseInt(userReply);
    if (ticketSelector >= 0 && ticketSelector < results.get(index).size()) {
      ticketUser.purchaseTicket(results.get(index).get(ticketSelector));
    } else {
      cmd.displayError("Invalid Ticket index, please try again");
    }
  } while (!allPurchased);
  cmd.print("All tickets purchased!");
}
```

Original image can be found on our Github here

The *prchTcks* function handles the selection and purchasing of tickets from pathfinding results. It first displays the results, (previously sorted), and allows for the selection of the preferred path (ticket list). Then it displays the tickets involved in the path with more detail, it once again allows for the Passenger to choose which tickets to purchase, automatically looping until the whole path is pruchased or the user quits explicitly. Utilizes the Passenger's pruchasing ticket method.
It implements UC

```java
public boolean purchaseTicket(Ticket ticket) {
    if (ticket == null) {
        userManagement.displayError("Invalid ticket information.");
        return false;
    }

    if (ticket.getOwner() != this) {
        userManagement.displayError("Ticket owner must match the authenticated passenger.");
        return false;
    }

    if (findTicketById(ticket.getId()) != null && ticket.isActive()) {
        userManagement.displayError("This ticket has already been purchased.");
        return false;
    }

    String transactionDescription = "Purchase ticket: " + ticket.getDescription();
    boolean success = userManagement.processTransaction(transactionDescription);

    if (!success) {
        userManagement.displayError("Ticket purchase failed. Please try again.");
        return false;
    }

    ticket.setStatus(Ticket.TicketStatus.ACTIVE);
    ticket.addHistoryEntry("PURCHASED");

    if (findTicketById(ticket.getId()) == null) {
        tickets.add(ticket);
    }

    String subject = "Ticket purchase completed";
    String body = "Hello " + getUsername() + ",\n\n" +
        "Your ticket purchase was successful.\n" +
        "Ticket: " + ticket.getDescription() + "\n\n" +
        "Thank you for using DispatchNet.";

    userManagement.sendEmail(new Email(getEmail(), subject, body));
    userManagement.print("Ticket purchase completed successfully.\n");
    return true;
}
```

Original image can be found on our Github here

The *PurchaseTicket* function allows a Passenger (and only a Passenger) to pruchase a ticket. First it ensures the ticket information is valid (I.E. non-null and not intestated to someone else), then it further checks the ticket has not been purchased already, to avoid posssible double-charge scenarios. The ticket is then added to the passenger's history and a confirmation email is sent. Lastly, the function returns "true" to confirm a successful transaction; It implements UC9

```java
private TrainType selectOrCreateTrainType() {
    var trainTypes = Main.getServiceManagement().getTrainTypes();

    if (!trainTypes.isEmpty()) {
        userManagement.print("Existing train types:\n");
        for (var entry : trainTypes.entrySet()) {
            userManagement.print("  [" + entry.getKey() + "] "
                + entry.getValue().getIdentifier()
                + " – seated: " + entry.getValue().getSeatedCapacity()
                + ", standing: " + entry.getValue().getStandingCapacity() + "\n" );
        }

        String choice = userManagement.prompt(
            "Enter an existing train type identifier to use it, or 'new' to create one:"
        );

        if (!choice.equalsIgnoreCase("new")) {
            var found = Main.getServiceManagement().getTrainType(choice);
            if (found.isPresent()) return found.get();
            userManagement.displayError("Train type not found. Creating a new one instead.");
        }
    } else {
        userManagement.print("No train types exist yet. Please create one.\n");
    }

    // fall through to creation
    return createTrainType();
}
```

Original image can be found on our Github here

The *selectOrCreateTrainType* function is responsible for handling of *TrainType* class. It does so by first allowing the user to either select an already existing one or to create a new one.
It implements UC29

```java
public void deleteJunction(String junction) {
    // Delete a junction (identified by junction id)
    var jct = network.getJunctions().get(junction);

    // junction not found (get returns null if id not found)
    if (jct == null) {
        userManagement.displayError("Junction not found");
        return;
    }

    // check if junction is in use (services list non-empty)
    if (jct.getServiceSets().size() > 0) {
        userManagement.displayError(
            "Junction is used by services and cannot be deleted"
        );
        return;
    }

    // attempt deletion and handle return values
    var res = network.removeJunction(junction);

    if (res.isPresent()) {
        // if present, deletion failed: check reason
        if (res.get() == Network.EditError.IN_USE) { // in use
            userManagement.displayError(
                "Junction is used by services and cannot be deleted"
            );
        }
        else { // not found
            userManagement.displayError("Junction not found");
        }
    }
    else { // success
        // print confirmation
        userManagement.print("Junction deleted: " + junction+ "\n");
    }
}
```

Original image can be found on our Github here

The *deleteJunction* function handles the smooth deletion of junctions, by first guaranteeing that no currently-in-use junctions are modified, then invoking the junction's internal removal code.
It implements UC27

```java
private Junction insertJunctionData() {
    // Loop until valid input is provided
    while (true) {
        // Prompt for junction name
        String name = userManagement.prompt("Enter junction name:");

        // check name uniqueness among junctions
        boolean nameTaken = network.getJunctions().values().stream()
            .anyMatch(j -> j.getName().equals(name));

        // if name is taken, show error and restart loop
        if (nameTaken) {
            userManagement.displayError(
                "Name already in use by another junction"
            );
            continue;
        }

        // prompt for coordinates
        String latS = userManagement.prompt("Enter latitude (float):");
        String lonS = userManagement.prompt("Enter longitude (float):");
        // parse coordinates and handle invalid input
        float lat, lon;
        try {
            lat = Float.parseFloat(latS);
            lon = Float.parseFloat(lonS);
        } catch (NumberFormatException e) {
            userManagement.displayError(
                "Invalid coordinates"
            );
            continue;
        }

        // build and return Junction without station data
        return new Junction(
            new GeoCoordinate(lat, lon), java.util.Optional.empty(), name,
            null, null
        );
    }
}
```

Original image can be found on our Github here

The *insertJunctionData* function allows the Network Manager to create interal Junction data for new junctions. First, it ensures the name is unique, then it allows input for the Junction's latitude and longitude, and lastly it invokes the Junction constructor.
It implements UC18

2.13 Team roles

We divided the project into 3 natural modules, so that everyone worked on every aspect of software engineering flow. Following, specific roles and activities, as well as the workload distribution.

Table 2.1: Roles and Activities

Team member	**modules**	**Roles and Activities**
Alessio Zanon	Path finding, Info view, Ticketing	Latex expert
Leonardo Tonetta	User authentication and management, Ticketing	Team leader: monitoring deadlines, submitting deliverables, scheduling meeting
Luca Fiorini	Network and Service management	Railway expert

Table 2.2: Workload distribution

Team member	**D1**	**D2**	**D3**
Alessio Zanon	34	33	33
Leonardo Tonetta	33	34	33
Luca Fiorini	33	33	34